SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	*Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.*		
Student Name:	K.Kabilan	**Reg. No.:**	192321157
Section / Batch:	B.Tech-IT	**Date:**	26/08/2026

Code of Conduct

I K.Kabilan (192321157) *(Name / Reg No)*

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K.Kabilan

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
	GRAND TOTAL:	**100 Marks**	

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
   a) Clustering effect
   b) Memory usage
   c) Performance under high load factor
   d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
   a) Clustering issues
   b) Collision resolution efficiency
   c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
   a) Practical significance
   b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
   a) Space complexity (stack usage)
   b) Ease of implementation
   c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

C04- AT3-

Comparative Analysis

Name: K. Kabilan
Reg. no: 192521157
Course: Data structures
Course code: CSA0306
Date: 18/03/2026

1) Compare linear probing and separate chaining for collision handling in hashing based on: a) clustering effect b) memory usage c) performance under high load factor d) which method is more suitable for large-scale application

Factor	Linear probing	Separate chaining
clustering effect	suffers from	Less clustering
memory usage	uses less memory because no extra linked lists	uses more demo pointers/ nodes
High load factor	performance decreases quickly	performs better
Large-scale applications	Less suitable when Table becomes full	more suitable because it handles collisions and high load.

conclusion: separate chaining is generally is preferred for large-scale applications because works efficiently even at Higher load factors

2) Compare quick sort and merge sort for sorting a linked list of 10 million nodes. Analyze memory requirements, cache behaviour, and Time complexity.

FOR a linked list of 10 million nodes.

Factor	Quick sort	Merge sort
Time complexity	Average: $O(n \log n)$, worst: $O(n^2)$	Best / Average / Worst $O(n \log n)$
Extra memory	$O(\log n)$ array stack	$O(\log n)$ recursion stack
Linked-list access	Not efficient because partitioning is difficult	very efficient
Cache behaviour	Better for arrays, but not ideal for linked lists	sequential access suits linked lists
Stability	usually not stable	Stable

Conclusion:

↳ merge sort is preferable for large linked lists because it provides $O(n \log n)$ Time, requires less extra space, and works efficiently.

↳ For arrays, Quick sort is generally preferable because it is in place, cache-friendly and fast in pratice.

Compare Quadratic probing and Double Hashing in terms of:

a) Clustering issues
b) Collision resolution efficiency
c) suitable for dense hash Tables.

Factor	Quadratic probing	Double hashing
clustering	Reduces primary clustering but can have secondary clustering.	Avoids primary and secondary clustering better
Collision resolution	Good	More efficient
Dense hash Tables	performance decreases at high load.	Better for dense Tables.
probe sequence	use quadratic function	uses a secondary hash function.

Conclusion: Double Hashing is Generally better because it provide a more uniform distribution of probs and works better when the hash Table is has highly loaded.

4) Compare Best-Case and worst-Case Time Complexity of sorting algorithm based on a) practical significance b) Impact on algorithm.

Factor	Best case	worst Case
Meaning	Minimum Time required	Maximum Time required
practical significance	shows ideal performance	shows possible poor performance
Algorithm selation	useful when input is usually favourable	more important for reliability
Example	Quick sort: $O(n \log n)$	Quick sort: $O(n^2)$

Conclusion: Best-Case Complexity shows the minimum time an algorithm may take, while worst-Case Complexity shows the maximum time. For large and unpredictable inputs, algorithm with better worst-case performance are Generally prefered.

pare Recursive and Iterative implementation of Quick in terms of:

a) space complexity
b) ease of implementation
c) Risk of stack overflow answer for these questions

Factor	Recursive Quick sort	Iterative Quick sort
space complexity	O(log n) average stack O(n) worst case	uses are explicit stack
Ease of Implementation	Easy And simple	more complex
stack overflow	Higher risk for bad paritions	Lower risk because stack can be controlled
Code	shorter	Longer

L> Recursive Quick sort is easier to implement but may cause stack overflow for large or unbalanced inputs. Iterative Quick sort avoids this risk by using an explicit stack.

L> Iterative Quick sort is safer for big large datasets, while recursive Quick sort is simple to implement.